

Enterprise Offline-First CRM

Production Architecture & Security Blueprints

Target Ecosystem:	Flutter Enterprise Cross-Platform (iOS & Android)
Architecture:	Clean Architecture + BLoC + Layered Storage Persistence
Integration Engine:	SAP Core API Wrapper with Background Isolation & Sync Routing
Security Level:	Military Grade At-Rest Crypto (AES-256 via SQLCipher) + Runtime Obfuscation
Document Version:	2026.1.0

1. High-Level System Architecture

This technical specification sets out the architectural foundation for an enterprise-level, offline-first Customer Relationship Management (CRM) mobile application. The platform relies on split storage layers, decentralized processes, and robust runtime isolation to completely eliminate data collision, security gaps, and interface blocking.

The Core Principle: UI-Abstraction

The Presentation layer (BLoC) maintains total isolation from both local storage systems and remote endpoints. It maps incoming application flows strictly to the Domain Layer Repository abstractions. The underlying data layers interact deterministically to execute state changes, storage writes, and remote data validation asynchronously.

2. The 4-Layer Persistence Matrix

To safely govern business entities, transactional inputs, application states, and configuration variables, the persistence footprint is explicitly segmented across four specialized storage systems:

Layer / Stack	Technology	Data Entities Governed	Security Implementation
Layer 1: Primary Business Engine	Drift (SQLite Engine)	Customers, Contacts, Leads, Price Books, Sales Orders, Quotations, Carts, Stock Counts, Route Visits, Sync Queue, and Audit Logs.	SQLCIPHER AES-256 Encrypted locally dynamically.
Layer 2: App & UI Preferences	Hive	Theme settings, locale parameters, feature flags, UI cache configurations, sorting/filter metrics, and dashboard layout.	Plaintext. No sensitive parameters permitted.
Layer 3: Cryptographic Secrets	flutter_secure_storage	OAuth Access/Refresh tokens, SAP Session IDs, Device IDs, Biometric Salts, and the Local DB Encryption Key.	HARDWARE SEALED iOS Keychain / Android Keystore.
Layer 4: Unbounded Binary Media	Native File System	Captured Visit Photos, Customer Attachments, Generated Quotation PDFs, and Field Signatures.	App-Sandbox restricted directory paths. Only string references stored in Layer 1.

3. Production Security & Environment Strategy

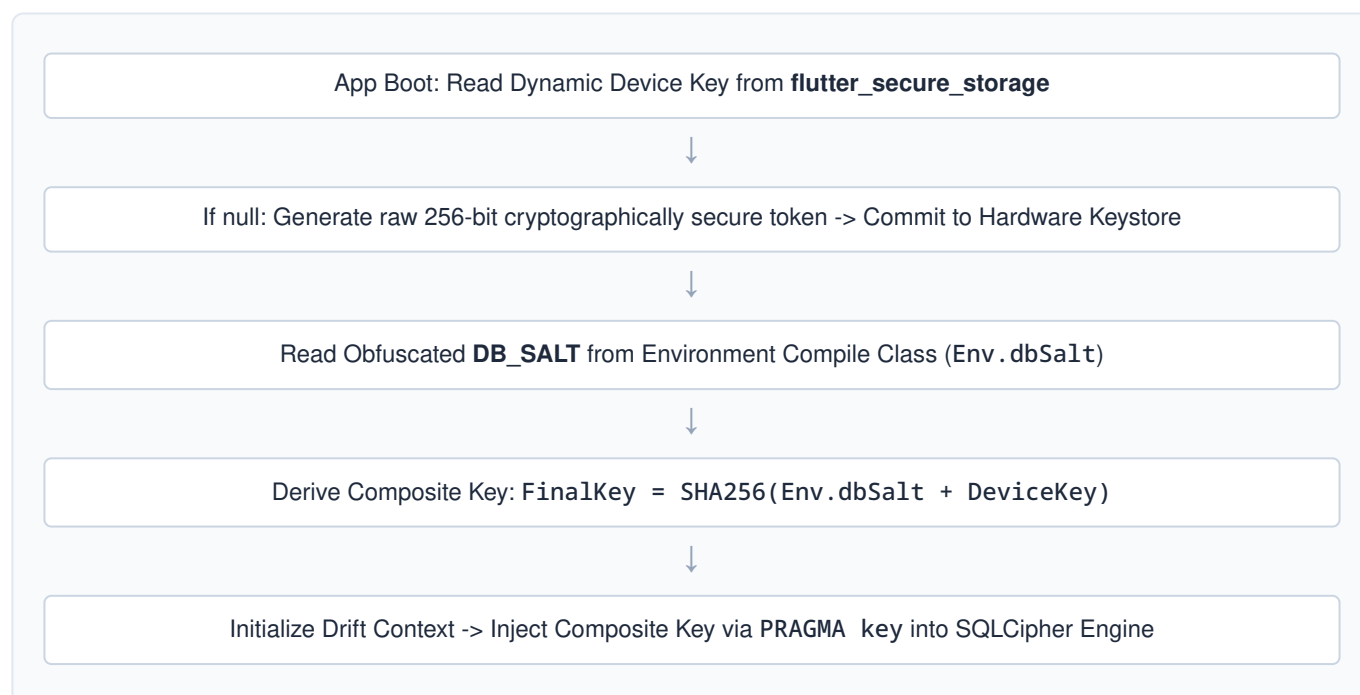
Config Isolation via Envied

To safeguard environment configurations without introducing plain-text liability within production builds, variables are obfuscated into compiled Dart byte-code via `envied`. Static environment variables like endpoints are structurally separated from dynamic device keys.

```
// Root definitions for git-ignored environment assets
// .env.dev and .env.prod config formats
SAP_API_URL=https://prod.sap-gateway.company.internal/api/v2
DB_SALT=e7b4c92a10dcf85b4138e93d9a74fe
```

Dynamic Cryptographic Key Derivation Flow

To protect business information if a device is stolen, the database encryption key must never be static. The database initialization sequence derives unique keys at runtime:



4. Resumable State Modeling (WorkflowSession)

To guarantee complete state recovery across aggressive operating system resource purges or absolute battery death events, the application employs a highly structural state preservation model via the `WorkflowSession` engine.

Workflow Session Entity Definition

```
class WorkflowSession extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get workflowOrigin => text();           // e.g., 'ROUTE_VISIT', 'MANUAL_ORDER'
  TextColumn get workflowType => text();             // e.g., 'QUOTATION', 'STOCK_COUNT'
  TextColumn get currentScreen => text();            // UI Route targeting
  TextColumn get currentStep => text();              // Stepper step indicator
  IntColumn get customerId => integer().nullable();
  IntColumn get cartId => integer().nullable();
  TextColumn get navigationArguments => text().nullable(); // JSON serialized route metadata
  TextColumn get status => text();                  // 'ACTIVE', 'SUSPENDED', 'SYNCED'
  DateTimeColumn get updatedAt => dateTime();
}
```

On initialization, the application's routing module checks for active sessions. If found, it routes the user back to the exact screen, reloading the state from the active transaction context.

5. Network Independence & Sync Architecture

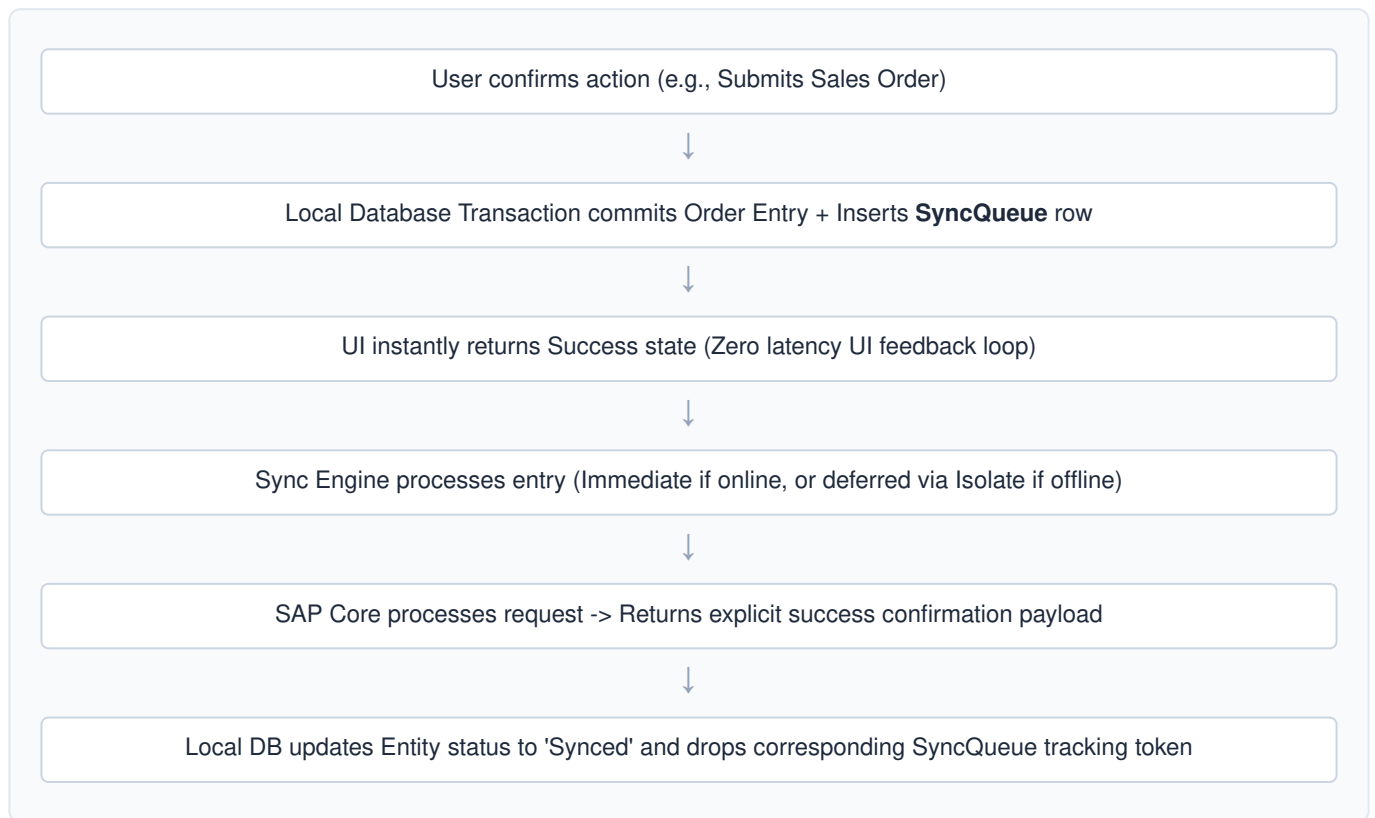
Reactive Network Interception

The connectivity footprint uses `connectivity_plus` to monitor the device's internet connection status. The application treats network drops as a standard operational state rather than a critical failure.

UX Architecture Note: When the app shifts to an offline state, blocking dialog alerts are strictly avoided to ensure uninterrupted workflows. Instead, a persistent, global status indicator smoothly shifts into view, signaling that operations will be stored securely on the device and synchronized automatically later.

Sync Engine & Background Processing

All mutations written to the database generate a parallel instruction entry in the `SyncQueue` table within a single local transaction.



Conflict Resolution Policy: By default, enterprise CRM data integration adopts a "Server Validation / Merge Rule Strategy". If a pricing check or inventory variance occurs while a user is offline, SAP marks the transaction for review, and the Sync Engine routes the item to an app-level "Action Required" dashboard queue. This prevents corrupted data from automatically overwriting system information.

6. Production Project Layout

The implementation follows Clean Architecture strict standards, maintaining total decoupled modular boundaries:

```
lib/
├── core/
│   ├── config/
│   │   ├── env.dart           // Envied configuration base class
│   │   └── env.g.dart         // Obfuscated compile-time definitions
│   ├── database/
│   │   ├── drift/
│   │   │   ├── app_database.dart // Global SQLCipher Drift abstraction
│   │   │   ├── tables/           // Isolated entity schema declarations
│   │   │   └── daos/             // Data Access Objects (Scoped accessors)
│   │   ├── hive/
│   │   │   └── preferences_box.dart // Lightweight rapid UI preferences
│   │   └── secure/
│   │       └── dynamic_key_store.dart // Keystore/Keychain interfacing
│   ├── network/
│   │   ├── connectivity_service.dart // Continuous stream monitoring
│   │   └── sap_client.dart           // Authenticated Axios-style HTTP wrappers
│   └── sync/
│       ├── sync_engine.dart         // Isolate-based background runner
│       └── conflict_manager.dart     // Conflict resolution strategies
├── features/
│   ├── customer/
│   ├── orders/
│   ├── visits/
│   └── workflow/
│       ├── presentation/           // BLoC definitions and UI layout components
│       ├── domain/                 // Strict functional entities & Use Case handlers
│       └── data/                   // Repositories mapping Local/Remote operations
└── main.dart                       // Absolute early bootstrap context entry point
```

7. Structured Implementation Strategy

The architecture rollout follows a structured phase plan to reduce refactoring impact and safely introduce security parameters:

1. **Phase 1: Cryptographic Foundation** — Implement envied obfuscation, establish hardware key storage integration protocols, and instantiate the SQLCipher encrypted Drift database container layer.
2. **Phase 2: Relational Data Schema Setup** — Implement standard data schemas inside Drift. Build efficient, high-performance DAOs to handle heavy offline queries smoothly.

3. **Phase 3: Resumable Workflow Management** — Connect the global routing configuration directly to the `WorkflowSession` table, enabling automatic data recovery and session resumption on app launch.
4. **Phase 4: Sync Pipeline Development** — Set up background processing isolates using native background workers. Implement robust transactional safety rules for queue mutations and SAP error handling.
5. **Phase 5: Performance Optimization** — Abstract large binary file streaming off the database layer, refine background task scheduling to conserve system resources, and complete end-to-end integration testing under simulated network blackouts.